

SQL Avançado

Window Functions (Funções de Janela)

Conceito Detalhado e Fundamentação

Diferente das agregações tradicionais com GROUP BY — que agrupam e reduzem o número de linhas retornadas —, as **Window Functions** realizam cálculos sobre um conjunto de linhas (chamado de **janela** ou *window*) relacionado à linha atual, **sem reduzir ou colapsar as linhas do resultado**.

Cada linha da consulta preserva sua identidade individual, recebendo colunas adicionais com os resultados calculados sobre a janela.

A Cláusula OVER()

A cláusula OVER() define os limites e a ordenação da janela de dados:

FUNÇÃO() OVER(PARTITION BY coluna_particao ORDER BY coluna_ordenacao)

1. **PARTITION BY**: Divide o conjunto de resultados em partições independentes (análogo ao GROUP BY, mas sem colapsar as linhas). Se omitido, a janela será a tabela/resultado inteiro.
2. **ORDER BY**: Define a ordenação lógica dos dados *dentro de cada partição*. É obrigatório para funções de classificação e navegação.

Funções de Classificação (Ranking)

- **ROW_NUMBER()**: Atribui um número inteiro sequencial único e estrito a cada linha dentro da partição, sem repetições (1, 2, 3, 4...).
- **RANK()**: Atribui a posição relativa. Em caso de empate nos valores do ORDER BY, atribui o mesmo número aos empatados e **salta** os números subsequentes (ex: 1, 2, 2, 4...).
- **DENSE_RANK()**: Atribui a posição relativa. Em caso de empate, atribui o mesmo número, mas **não salta** posições (ex: 1, 2, 2, 3...).

Funções de Navegação (Offset Functions)

- **LAG(coluna, offset, default)**: Acessa o valor de uma coluna presente em uma linha **anterior** dentro da mesma partição. Muito utilizada para calcular variações diárias/mensais ($V_{atual} - V_{anterior}$).
- **LEAD(coluna, offset, default)**: Acessa o valor de uma coluna em uma linha **posterior** dentro da mesma partição. Utilizada para comparações futuras ou tempo até o próximo evento.

Funções Agregadoras com Janela

Ao utilizar SUM(), AVG(), COUNT(), MIN() ou MAX() acompanhadas da cláusula OVER(ORDER BY ...), o SQL calcula **totais acumulados (Running Totals)** ou **médias móveis**

CTEs Avançadas e CTEs Recursivas

Conceito Detalhado e Fundamentação

Uma **CTE (Common Table Expression)** é um conjunto de resultados temporário e nomeado, definido no escopo de execução de uma única instrução SELECT, INSERT, UPDATE ou DELETE. Ela é declarada usando a cláusula WITH.

Vantagens das CTEs em Relação a Subconsultas (*Subqueries*):

1. **Legibilidade e Reutilização:** Permite dividir consultas altamente complexas em blocos lógicos modulares legíveis de cima para baixo.
2. **Referência Múltipla:** Uma mesma CTE pode ser consultada várias vezes dentro da consulta principal.
3. **Recursividade:** Capacidade de navegar em estruturas hierárquicas e em árvore.

CTEs Recursivas

Uma **CTE Recursiva** é uma consulta que faz referência a si mesma repetidamente até atingir uma condição de parada. É a solução padrão do SQL para percorrer **estruturas hierárquicas de profundidade indeterminada** (ex: organogramas de empresas, categorias/subcategorias, grafos de dependência).

Otimização e Performance

Conceito Detalhado e Fundamentação

Otimizar consultas SQL envolve entender como o SGBD processa o código declarativo e o transforma em um **Plano de Execução Físico**.

Plano de Execução (Execution Plan)

O **Otimizador de Consultas (Query Optimizer)** do SQL Server avalia diversas maneiras de processar uma consulta e escolhe a de menor custo computacional estimado (baseado no consumo de CPU e I/O de disco).

Operações Críticas a Identificar no Plano de Execução:

- **Table Scan:** O mecanismo lê **todas as páginas de dados da tabela inteira** linha por linha. Indica falta de índices adequados.
- **Clustered Index Scan:** Lê a tabela inteira organizada pelo índice clustered. Pode ser tão custoso quanto um Table Scan se a tabela for grande.

- **Clustered Index Seek / Index Seek:** O mecanismo navega na estrutura em **Árvore B (B-Tree)** do índice e vai diretamente para as linhas desejadas. **É o cenário de alta performance ideal.**
- **Key Lookup:** Ocorre quando um índice *Non-Clustered* é usado para localizar a linha, mas ele não contém todas as colunas solicitadas na consulta SELECT. O banco precisa realizar uma busca adicional na tabela principal para buscar o restante dos campos.

Arquitetura de Dados moderna & Paradigmas

Evolução Arquitetural: Data Warehouse → Data Lake → Lakehouse

Conceito Detalhado

- **Data Warehouse (DW):** Estrutura tradicional (ex: Teradata, SQL Server DW, Snowflake). Exige dados altamente estruturados (*Schema-on-Write*), focados em SQL/BI, com alto custo de armazenamento e computação acoplados.
- **Data Lake:** Repositório central para dados estruturados, semi-estruturados (JSON) e não-estruturados (vídeos, logs). Permite armazenamento barato (Object Storage como AWS S3, Azure Blob, GCP GCS) e *Schema-on-Read*. **Gargalo:** Risco de virar um "*Data Swamp*" (pântano de dados) sem governança e sem suporte nativo a transações ACID.
- **Data Lakehouse:** Consolidação do mercado. Combina o custo reduzido e flexibilidade do Data Lake com as garantias **ACID** (Atomacidade, Consistência, Isolamento e Durabilidade), versionamento de dados (*Time Travel*) e performance de leitura do Data Warehouse através de formatos de tabela abertos (**Delta Lake, Apache Iceberg, Apache Hudi**).

Característica	Data Warehouse	Data Lake	Data Lakehouse
Estrutura de Dados	Apenas Estruturado	Estruturado, Semi e Não-estruturado	Todos os tipos de dados
Garantia ACID	Sim (Nativo)	Não (Manual / Complexo)	Sim (Via Delta/Iceberg)
Paradigma de Esquema	Schema-on-Write	Schema-on-Read	Schema Enforcement & Evolution
Custo de Storage	Alto	Baixo (Cloud Object Storage)	Baixo (Cloud Object Storage)

Paradigmas de Processamento: Batch vs. Streaming

Conceito Detalhado

- **Processamento em Lote (Batch):** O dado é acumulado em blocos durante um intervalo de tempo (ex: a cada 1 hora, diariamente às 00:00) para processamento em massa.
 - *Casos de uso:* Fechamento financeiro mensal, carga noturna de relatórios executivos.
- **Processamento em Tempo Real (Streaming / Near Real-Time):** O dado é processado registro a registro (ou em micro-batches de segundos) à medida que o evento ocorre na origem.
 - *Casos de uso:* Detecção de fraude em cartão de crédito, rastreamento de entregas via GPS, monitoramento de sensores IoT.

Workloads e Consumo de Dados

- **Workloads Analíticas (BI/Dashboards):** Requerem baixíssima latência de consulta, agregação pré-calculada e dados modelados (Star Schema).
- **Consultas Ad-hoc:** Usuários avançados e analistas explorando grandes volumes sem estrutura fixa.
- **Machine Learning / Data Science:** Requerem acesso a dados brutos e semi-estruturados, histórico não-agregado e leitura massiva de arquivos Parquet/ORC.

Módulo 4: Qualidade & Validação de Dados

As 6 Dimensões Essenciais da Qualidade

1. **Compleitude:** O dado necessário está presente? (Ausência de NULLs indevidos).
2. **Acurácia:** O dado reflete a realidade da operação? (Ex: preço de produto positivo).
3. **Consistência:** Os dados dizem a mesma coisa em diferentes tabelas/sistemas? (Ex: valor total do pedido bate com a soma dos itens).
4. **Tempestividade (Timeliness):** O dado está atualizado a tempo para ser útil à tomada de decisão?
5. **Unicidade:** O mesmo evento/entidade não está duplicado no repositório.
6. **Validade:** O dado obedece ao formato, domínio ou intervalo esperado (ex: e-mail válido, UF com 2 letras).

Testes Automatizados e Data Contracts

No desenvolvimento moderno de software e engenharia de dados, a qualidade é tratada como **código**:

- **Data Contracts (Contratos de Dados):** Acordos formais entre quem produz o dado (sistemas operacionais) e quem consome o dado (Engenharia de Dados). Se a origem alterar a estrutura (schema) sem aviso, o pipeline bloqueia a ingestão.
- **Frameworks de Qualidade:** Apresentação conceitual do funcionamento de ferramentas como **Great Expectations** e **dbt test**.

Módulo 5: Arquitetura Medalhão (Medallion Architecture) (3h)

Visão Geral do Padrão Medalhão

A Arquitetura Medalhão (popularizada pela Databricks) organiza os dados dentro de um Data Lakehouse em **três camadas lógicas de refinamento gradual**

Detalhamento e Construção das Camadas no VS Code

Crie o arquivo arquitetura_medalhao.sql no VS Code.

Camada Bronze (Raw / Landing Zone)

- **Objetivo:** Armazenar os dados exatamente como vieram da fonte (Sistemas OLTP, APIs, arquivos JSON), sem alteração de valores.
- **Padrão:** Ingestão incremental no formato *Append-Only* com inclusão de metadados de controle (Data_Ingestao, Nome_Arquivo_Origem).

Camada Silver (Cleansed / Enriched)

- **Objetivo:** Oferecer uma visão limpa, tipada e padronizada de toda a empresa.
- **Ações:** Aplicação dos tipos de dados corretos (castings), remoção de duplicatas, tratamento de NULLs, padronização de textos (ex: converter estado "sp" para "SP" uppercase) e validação de contratos de dados.

Camada Gold (Curated / Business)

- **Objetivo:** Entregar os dados totalmente modelados e otimizados para o consumo final pelas áreas de negócio (Power BI, Tableau, Modelos de Machine Learning).
- **Ações:** Aplicação das regras de negócio, agregações estratégicas, criação das Tabelas **Fato e Dimensão** (Star Schema).

Arquitetura de Dados Moderna & Paradigmas (3 Horas)

Evolução Arquitetural: Data Warehouse → Data Lake → Data Lakehouse

Conceituação Teórica Detalhada

- **Data Warehouse (DW Traditional):**

- **Conceito:** Repositório central de dados estruturados provenientes de sistemas operacionais (OLTP).
- **Arquitetura:** Baseia-se no modelo *Schema-on-Write* (o esquema das tabelas precisa ser rigorosamente definido e validado **antes** da gravação do dado).
- **Vantagens:** Altíssima performance para consultas SQL e BI; garantia nativa de transações ACID.
- **Limitações:** Dificuldade para escalar com dados não-estruturados (JSON, logs, imagens); alto custo de armazenamento e computação acoplados.

- **Data Lake:**

- **Conceito:** Repositório escalável que armazena grandes volumes de dados no seu formato bruto (*Raw*), sejam eles estruturados, semi-estruturados ou não-estruturados.
- **Arquitetura:** Baseia-se em armazenamento de objetos em nuvem (*Cloud Object Storage* como AWS S3, Azure Blob Storage, GCP Cloud Storage) sob o paradigma *Schema-on-Read* (a estrutura é interpretada apenas no momento da leitura/consulta).
- **Vantagens:** Baixíssimo custo de armazenamento; suporte total a ingestões em massa e Machine Learning.
- **Limitações:** Sem governança estrita, corre o risco de virar um **Data Swamp** (pântano de dados); falta de suporte nativo a operações de atualização/deleção (UPDATE/DELETE) e falta de suporte a transações ACID simultâneas.

- **Data Lakehouse:**

- **Conceito:** O estado da arte da arquitetura de dados moderna. Une a flexibilidade, o baixo custo e o armazenamento ilimitado do Data Lake com as garantias de consistência, performance e governança do Data Warehouse.
- **Habilitador Tecnológico:** Camadas de armazenamento baseadas em formatos de tabela abertos (*Open Table Formats* como **Delta Lake**, **Apache Iceberg** e **Apache Hudi**).
- **Recursos-Chave:**
 - **Transações ACID:** Leitura e escrita simultâneas sem corromper arquivos.
 - **Time Travel (Versionamento):** Capacidade de consultar o estado exato dos dados em momentos do passado.

- **Schema Enforcement & Evolution:** Impede que dados fora do padrão corrompam a tabela, mas permite evolução controlada de colunas.

Paradigmas de Processamento e Workloads

Batch vs. Streaming (Near Real-Time)

Atendimento às Workloads do Negócio:

1. **Workloads Analíticas (BI/Dashboards):** Exigem dados limpos, consolidados, modelados (Star Schema) e com baixa latência de consulta.
2. **Consultas Ad-hoc:** Exigem ferramentas de consulta direta em arquivos (como Trino, Athena ou Databricks SQL) sobre dados semi-estruturados.
3. **Data Science & Machine Learning:** Exigem dados em grão fino (sem agregação), histórico completo de interações e integração com frameworks Python/Spark.

Qualidade & Validação de Dados

As 6 Dimensões Práticas da Qualidade de Dados

1. **Completeness:** Ausência de registros ou atributos vazios (NULL) onde a informação é obrigatória.
2. **Accuracy:** O dado reflete com precisão a realidade do evento de negócio (ex: preço de venda > 0).
3. **Consistency:** Ausência de contradições entre dados em diferentes tabelas ou sistemas (ex: valor total do pedido == soma dos seus itens).
4. **Timeliness (Timeliness):** O dado está disponível no tempo esperado para a tomada de decisão.
5. **Uniqueness:** Ausência de duplicidades involuntárias para a mesma chave/entidade.
6. **Validity:** O dado está em conformidade com formatos ou domínios predefinidos (ex: e-mail válido, UF com 2 caracteres, CPF válido).

Data Contracts (Contratos de Dados)

Os **Data Contracts** são um acordo de nível de serviço (SLA) de software entre os Engenheiros de Software (que produzem os dados nos bancos operacionais) e a Equipe de Dados (que ingere esses dados).

Estrutura Típica de um Contrato de Dados:

- **Garantia de Schema:** Nome do campo, tipo exato (INT, VARCHAR), obrigatoriedade (NOT NULL).
- **Garantia de Semântica:** A definição exata do que o campo representa (ex: status_venda só pode assumir 'PAGO', 'CANCELADO', 'PENDENTE').
- **Garantia de SLA:** Horário limite de entrega da carga diária.